

SLIM-ARC 项目全景总结（答辩理解版）

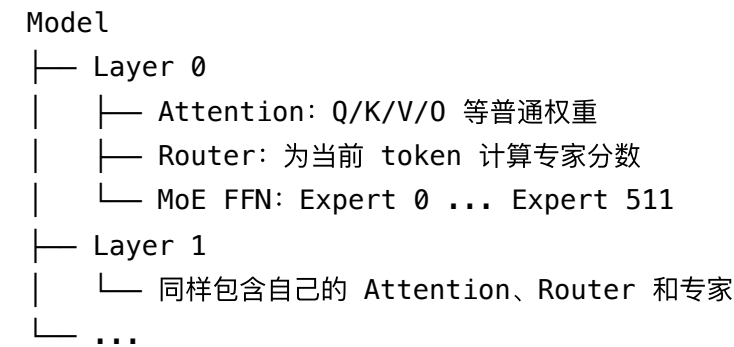
核心一句话：SLIM-ARC 不改变模型权重、原生 Router 选择和生成结果，而是把 MoE 的动态稀疏性转化为可控制的操作系统页面行为：运行时识别阶段、产生候选、执行预算准入和异步预取，再用真实 Router 结果结算命中与浪费，回收错误页、保留高复用页，并针对不同设备选择不同策略。

本文基于决赛技术报告、答辩 PPT、前期调研文档和 GitHub main 分支代码整理。文中的“**全新创新**”指本项目相对所采用基础机制与已有工作的原创增量；它不等同于已经完成严格学术查新的“全球首次”。

1. 先建立统一心智模型

1.1 模型、层、专家、Tensor 与 Weight

一个 Transformer 模型由很多 **Layer** 组成；每个 MoE Layer 内通常包括 Attention、Router、Shared Expert 和大量 Routed Experts：



因此是“**每层有多个专家**”，不是“一个专家有多个层”。一个 Expert 本质上是一套 FFN，通常包含 up/gate/down 等多个权重矩阵。

- **Tensor**：多维数组，是数据的存储与计算形式。
- **Weight**：在模型中承担参数角色的 Tensor。
- **普通权重**：Attention、Norm、Router、Embedding 等基本每次都要访问的权重。
- **专家权重**：只有被 Router 选中的 Routed Expert 才会访问，具有动态稀疏性。

GGUF 为便于存储，可能把 512 个专家的同类权重合并为一个三维 Tensor。SLIM-ARC 根据 Tensor 基址、总大小和专家数，计算某个专家切片的地址范围。

1.2 48.41 GB 为什么能在 2 GiB、no-swap 下运行

mmap 只把 GGUF 文件映射到虚拟地址空间，并不把 48.41 GB 权重一次性读入 RAM。CPU 首次访问某页时发生 page fault，内核再从模型文件把该页读入 page cache；内存紧张时，干净的文件页可以直接丢弃，未来再从 GGUF 读取。

48.41 GB GGUF
↓ mmap
48.41 GB 虚拟地址映射
↓ demand paging
约 2 GiB 当前物理工作集

这与 swap 不同：模型页本来就有 GGUF 作为 backing file，不需要先写入匿名 swap。因此 swap=0 仍允许 file-backed 页面被回收和重新加载。

真正决定极限可运行性的基础是：

1. **mmap / demand paging**：虚拟映射与物理驻留解耦；
2. **关闭 CPU repack**：避免为几十 GB 量化权重再建立大型匿名重排副本；
3. **cgroup/no-swap 硬约束**：让实验资源边界可复现。

2 GiB 表示模型运行时的物理内存边界，不代表模型被压缩成 2 GiB。

1.3 Prefill、Decode 与一次 Graph Compute

- **Prefill**：一次处理整段 Prompt，通常按层大范围扫描权重。
- **Decode**：每轮生成一个或少量 token，重复经过全部 Layer，MoE 专家访问更稀疏。
- **计算图**：由 MatMul、Norm、Attention、TopK、Expert FFN 等节点及其数据依赖组成的 DAG。
- **一次 graph_compute**：把当前 Prefill 或 Decode 的计算图真正在 ggml/CPU backend 上执行一遍。

Decode 会循环：

构建/执行 Decode Graph
↓
得到 logits 并采样新 token
↓
下一次 Decode Graph

2. SLIM-ARC 的分层系统设计

2.1 生存层：先让超大模型在小内存里成立

来源分类：基础机制 + 本项目工程集成

- 复用 POSIX mmap、page fault、page cache 和 cgroup。
- 构建时关闭 GGML CPU repack。
- 只在大模型、有效 mmap 且未禁用时创建 SLIM-ARC Runtime。
- 在模型加载后注册 mapping、普通 Tensor 和专家 Tensor 的地址、大小、Layer 与文件信息。

这一层首先回答“能不能跑”，不是“是否最快”。

2.2 普通权重：阶段与设备感知的有界异步预取

已有基础：FlexInfer 已提出 Layer-ahead Async Prefetch，即计算当前层时由 I/O 线程提前加载后续层权重，以计算覆盖 I/O。

本项目改进：SLIM-ARC 在普通异步预取上进一步加入：

- Layer 范围识别与前瞻窗口；
- Prefill/Decode 阶段识别；
- SEQUENTIAL/NORMAL/RANDOM 可配置页建议；
- Tensor 页范围向外对齐；
- 重叠页范围合并；
- 字节级内存/I/O Budget；
- 有界异步队列；
- 慢存储单 Worker、窗口 1、同层去重与 latest-wins；
- cgroup 压力下的动态收缩和安全降级。

RANDOM 与 SEQUENTIAL 的正确理解

它们不是“慢读”和“快读”，也不是“少占内存”和“立刻占满内存”，而是给内核的访问模式提示：

- RANDOM：抑制 readahead，减少附近无用页，但可能形成大量小粒度同步 page fault。
- SEQUENTIAL：允许内核合并顺序读和提前预读，减少后续阻塞，但可能读入少量未来不用的页。

理论上 Prefill 大范围扫描适合 SEQUENTIAL，而 MoE Decode 的专家访问较稀疏，最初认为 RANDOM 合理。但 Decode 并非只有专家：Attention、Norm、Router 等普通权重仍按 Layer 顺序访问。对整个大映射使用 RANDOM，可能为了抑制专家预读而同时破坏普通权重的顺序 I/O。

RK3588 实验证明静态 `RANDOM` 会把顺序访问碎片化为同步小缺页，`Prefill/Decode` 降至 `0.44/0.26 token/s`；阶段策略恢复至 `2.84/1.40 token/s`。因此最终结论不是“`Decode` 必须 `RANDOM`”，而是：

【**全新创新 1**】页访问策略必须同时由运行阶段、模型工作集比例和设备存储链路决定；**Runtime** 提供机制，设备策略选择 **SEQUENTIAL**、**NORMAL**、**RANDOM**、窗口大小或直接关闭预取。

2.3 MoE 专家：Router 反馈驱动的预测—结算—回收—驻留闭环

普通权重预取回答“下一层大概需要哪些 Tensor”；专家管理进一步回答“这一层的 512 个专家中到底值得搬哪些”。

候选信号

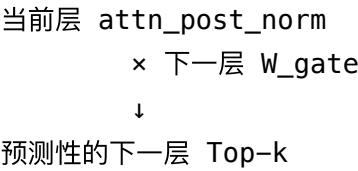
信号	含义	性质
Temporal	上一个 token 在同一 Layer 选择的专家	低成本预测
Inline Router	原生 <code>ffn_moe_topk</code> 一算出就通过 <code>callback</code> 读取	真实事实，不是预测
Cross-layer Gate	用当前层 Attention 后状态乘下一层 Gate 权重，近似预测下一层 Top-k	额外计算预测
Transition	在线统计“本层专家集合 → 下一层专家集合”的转移频次	无神经网络训练的历史预测
Popularity	根据长期热度补充 Top-K 热门专家	覆盖增强

Inline Router

Inline Router 不重新计算 Router，也不改变模型决策，只是比“整张 Graph 执行完再扫描”更早观察原生 Router 的真实结果。

Cross-layer Gate

真实下一层 Router 输入依赖当前层最终输出，因此不能提前得到完全准确的结果。项目使用当前层已经得到的 `attn_post_norm` 作为近似输入，提前执行：



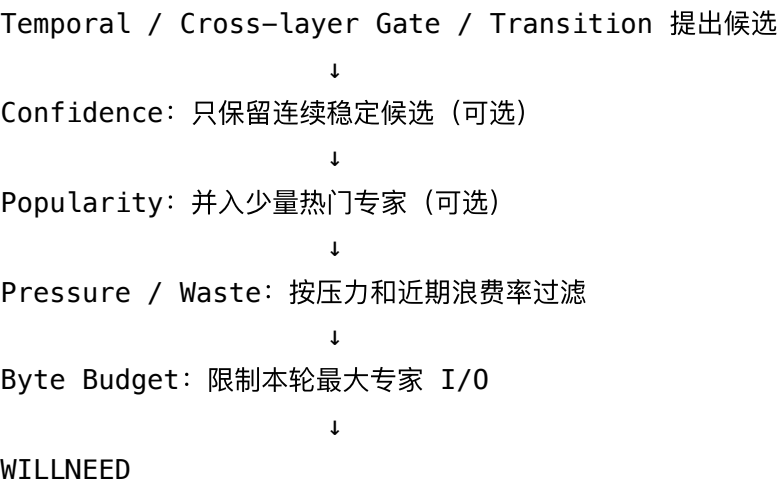
它与当前层 Expert FFN 形成时间重叠，但会消耗 CPU、Cache 和内存带宽，因此高命中率不等于端到端最快。

Transition

Transition Table 在线记录：当前 Layer 的某个 Source Expert 经常对应下一 Layer 的哪些 Target Experts；预测时累计多个 Source 行的 Target 计数并取 Top-k，老统计周期性衰减。

候选如何汇总

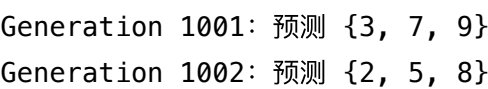
当前实现不是把所有预测器同时做神经网络 Ensemble，而是选择一条主要候选来源，再经过统一过滤流水线：



Cross-layer Gate 与 Transition 倾向互斥启用，避免同时产生两套高成本投机路径。若真实专家没有被预测到，模型仍通过普通 demand paging 加载，正确性不受影响，只是没能隐藏 I/O 延迟。

Generation Token 与一次性结算

异步系统中，同一个 Layer 可能连续产生多轮预测。SLIM-ARC 为每次专家预取分配单调递增的 Generation ID，相当于“投机事务号”：



真实 Router 结果只与相同 Generation 结算；成功 WILLNEED 的实际字节被分别计入 hit 和 waste，失败或缺失 Router 时只取消一次，避免轮次覆盖、重复记账或统计串台。

错误专家页安全回收

预测集合为 P 、真实选择为 S ，错误专家为 $P - S$ 。错误页可以用 `DONTNEED` 建议回收，但专家边界可能与相邻专家共享物理页，因此：

- `WILLNEED` 为覆盖 Tensor，页范围向外对齐；
- `DONTNEED` 为避免误伤，只回收完全位于错误专家内部的完整页，页范围向内对齐；
- 对专家数、整除性、ID、地址溢出和页大小严格检查，任何异常只跳过优化，不影响推理。

Shared/Hot Expert 驻留

- Shared Expert 每个 token 都执行，具有确定复用价值。
- Hot Expert 根据跨 token 稳定性、热度和最近性进入有限驻留集合。
- 先用 `mincore()` 检查页面已经 Resident，再 `mlock()`，避免为了“常驻”反而主动制造慢盘 I/O。
- Budget 满时可用 LRU 或 LFRU 选择 `munlock()` 的 Victim；Pi 实验中复杂 LFRU 增加扫描与维护成本，最终简单 LRU 更优。

【全新创新 2】项目将专家预测扩展为完整的 Router 反馈闭环：候选生成 → 字节预算准入 → Generation 精确归因 → 原生 Router 事实结算 → 错误内部页安全回收 → Shared/Hot 驻留 → 压力与浪费反馈。核心不只是“预测哪些专家”，而是“猜错后如何安全、可观测、可撤回”。

2.4 Pressure、Prediction Quality 与 Budget：防止优化过度

这三部分可以统一理解为三道刹车：

Pressure：现在还能不能做？

↓

Quality：最近预测值不值得做？

↓

Budget：本轮最多允许做多少？

内存压力

根据 `memory.current / memory.max` 分为：

- $<85\%$ ：Normal；
- $\geq 85\%$ ：High；
- $\geq 95\%$ ：Critical。

恢复时使用不同阈值和连续样本形成 hysteresis，避免在阈值附近反复震荡。Critical 基本拒绝 Routed Expert 驻留；High 只保留稳定候选。

预测浪费

以 $waste / (hit + waste)$ 形成 EWMA。长期浪费高时，只允许更稳定的候选；连续恢复后再逐渐放开。

统一预算

静态预算还要根据 cgroup 剩余空间收缩：

$$\text{有效预算} = \min(\text{静态预算}, \max(0, \text{memory.max} - \text{memory.current} - \text{reserve}))$$

随后按运行阶段在 Weight、KV、Expert 间分账。这里的“逐周期”不是 CPU 时钟周期，而是一次 Runtime Scheduler tick()，通常对应一次 Graph Execution / 生成 Step；下一轮会根据新的压力重新分配。

当前实现边界必须准确表述：

- Weight：已接入字节级硬准入；
- Expert：已接入字节级硬准入；
- KV：已有统一策略接口和独立模块，但尚未与 Model-owned Runtime 建立同等强度的逐轮 Hard Budget。

【全新创新 3】项目提出 **Pressure—Quality—Budget** 三层准入，将普通权重、专家投机和驻留放在统一资源视角中；预算的目标不只是“尽量利用带宽”，也包括证明某些投机 I/O 应当被拒绝。

2.5 KV Cache 与 Attention 协同

这一部分主要集成成熟方向，用于补足长上下文的另一条内存增长路径，不应表述为项目原创算法。

机制	做什么	代价/语义
KV q4_0	将 K/V 从高精度压缩到低比特	空间和带宽下降，但增加量化/反量化开销；短上下文可能变慢
StreamingLLM	保留开头少量 Attention Sink 和最近滑动窗口，驱逐中间旧 KV	将 KV 从 $O(L)$ 控制为近似 $O(S+W)$ ，但不再是完整历史 Full Attention

机制	做什么	代价/语义
FlashAttention	通过精确分块减少 Attention 中间量与内存流量	不删除历史 KV，不改变精确 Attention 数学结果

当前仓库中的 KV Eviction Manager 包含 Hot/Warm/Cold Block、Attention Score、滑动窗口和可选 mmap 临时文件 Offload 原型；其接线和物理释放成熟度低于 Weight/Expert 主路径，答辩应称为协同模块或研究原型。

2.6 生命周期安全：Runtime、Lease、Registry、Worker 与 Queue

这一部分不直接决定预测准确率，却决定异步优化是否安全。

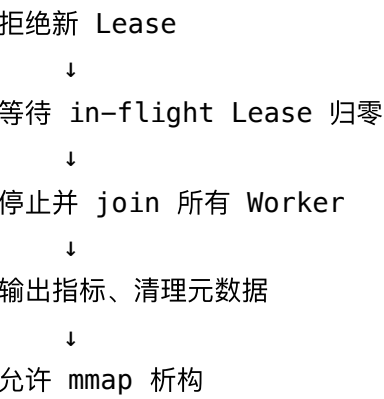
两种“页面不在了”

- 1. 页面被 **page cache** 淘汰，但 **mmap** 仍存在：地址合法，WILLNEED 可重新从 GGUF 读取。
- 2. 模型已经 **munmap**：地址不再属于模型，后台 Worker 再访问就是生命周期错误。

对象关系

- **Runtime Owner**：一个模型对应的 SLIM-ARC 控制器，持有 Scheduler、Worker、映射元数据、专家状态、预算和指标。
- **Worker**：后台处理 WILLNEED/DONTNEED 等页建议的线程。
- **Lease**：Graph 执行期间取得的临时使用权；Lease 存在时模型映射不能被销毁。
- **Registry**：让 Graph 执行路径找到当前 Active Runtime 的查找入口。
- **Bounded Queue**：Graph Thread 提交预取请求，Worker 异步领取；上限与 stale/latest-wins 防止旧请求无限堆积。

销毁顺序：



【全新创新 4（工程系统层）】项目将早期全局 Scheduler 改造成 Model-owned Runtime，以 Move-only Lease、Bounded Queue、锁内快照/锁外系统调用和确定析构顺序，把异步预取的地址有效性与模型生命周期绑定。

当前 Registry 仍主要服务单 Active Model；多模型并行 Runtime 是后续可扩展方向。

2.7 可复现工程与 Agent Harness

幂等集成与可观测性

- 独立 slim-arc-* C++ 模块；
- apply-slim-arc.py 基于固定 Anchor 幂等注入 llama.cpp，Anchor 缺失时 fail-fast；
- Baseline/Patched 独立构建和动态链接检查；
- 结构化 Runtime Metrics 记录 Weight/Expert 请求、发出、跳过、命中、浪费、回收、驻留和压力；
- 保存模型、源码、镜像、配置和原始日志身份；
- C++/Python 单测及 ASan/UBSan 路径。

这些机制将“能跑的原型”提升为可审计、可 A/B、能解释负优化的系统。

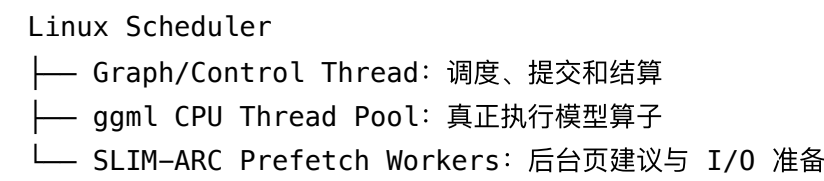
Agent Harness

Agent Harness 复用常驻 OpenAI-compatible 模型服务，控制上下文、输出 token、工具超时、白名单、工作目录和总步数，避免多轮 Tool Call 重新加载大模型或无限扩大上下文。它是上层资源协同扩展，不是 80B 模型 TPS 加速核心。

3. 从 GGUF 到生成答案的完整调用链

1. 打开 GGUF，读取模型结构、Tokenizer、Tensor 名称/形状/偏移/量化格式
2. 对模型文件建立 mmap，不整体读取 48 GB
3. 创建 ggml Tensor 描述符，其 data pointer 指向 mmap 地址
4. SLIM-ARC register_mapping：登记映射基址、大小和文件信息
5. register_tensor / register_expert_tensor：建立 Layer→Tensor→Expert 地址索引
6. 创建并 activate Model-owned Runtime
7. 用户文本 Tokenize 为 Token IDs
8. llama.cpp 构建 Prefill 或 Decode 计算图
9. graph_compute 前 acquire Lease、识别阶段、执行 unified tick
10. Graph/Control Thread 将未来 Layer 的预取请求放入有界 Queue
11. ggml CPU Compute Pool 执行 MatMul、Attention、Router、Expert FFN
12. SLIM-ARC Worker 并行执行页建议，计算与 I/O 尽量重叠
13. Inline Hook 捕获原生 Router Top-k，结算 Generation
14. 命中页进入复用/驻留，错误页按内部完整页 DONTNEED
15. 得到 logits, Sampler 选择新 Token；Decode 回到第 8 步
16. 任务结束后拒绝新 Lease、等待调用结束、join Worker、最后 munmap

这些线程是并行协作关系，不是简单父子层级：



4. 创新来源与完成度分类

机制	来源分类	在 SLIM-ARC 中的角色
mmap、page fault、 madvise、mlock、 mincore、cgroup	OS 基础机制	物理页控制与资源边界
Layer-ahead Async Prefetch、部分参数锁定	FlexInfer 等已有工作	普通权重预取基础
Q4_K_M/IQ4_XS、KV q4_0	已有量化技术	减少权重或 KV 字节
StreamingLLM、 FlashAttention	成熟外部方案	长上下文和 Attention 协同

机制	来源分类	在 SLIM-ARC 中的角色
LRU/LFRU	经典缓存策略	项目 Hot Expert Cache 的替换候选
阶段/ 设备感知页建议与负优化恢复	全新创新 1	决定不同设备该早读、少读还是不读
Router 反馈专家闭环	全新创新 2	预测、结算、回收、驻留和反馈统一
Pressure—Quality—Budget 三层准入	全新创新 3	防止多个优化模块共同挤爆资源
Model-owned Runtime + Lease + Bounded Queue	全新创新 4（工程层）	保证异步地址生命周期和并发安全
Cross-layer Gate / Transition Predictor	本项目预测原型与实现增量	提前产生下一层专家候选，研究收益与开销
幂等 Patcher、指标 Schema、证据哈希	本项目工程创新	可复现、可审计、可归因
Agent Harness	本项目集成扩展	让多轮 Agent 负载服从同一资源目标

5. 哪些机制真正决定结果

决定“能不能运行”

- 1. mmap / demand paging；
- 2. 关闭 CPU repack；
- 3. cgroup/no-swap 下的有界工作集。

决定强内存压力下“快不快”

- 1. 阶段与设备感知的页建议；
- 2. 普通权重的有界 Layer-ahead 预取；
- 3. 避免把顺序访问错误地拆成同步小缺页。

RK3588 3 GiB 下 Decode 从 0.70 提升至 2.21 token/s ，主要收益来自权重页路径，而非短工作负载中的 KV Eviction。

决定慢存储下“是否浪费 I/O”

- 1. 关闭无收益 Weight/Expert Speculation；
- 2. Shared/Hot Expert 只驻留确定复用、已 Resident 的页；
- 3. 严格缩小总预算。

Pi 上 Shared/Hot 驻留使 Decode 提升 14.16%、Wall Time 降低 6.58%；16 MiB Budget 让投机 Weight I/O 降低 98.44%，吞吐近似不变。

有研究价值但不是最终决定性主线

- Cross-layer Gate 命中率可达 88.56%，但额外 Gate MatMul 也占用 CPU 和内存带宽，未成为最低 Wall Time 方案；
- Wrong-page Reclaim 的安全机制完整，但 Mac 正式矩阵中 Reclaim 调用为 0，不能把性能变化归因于 DONTNEED；
- KV Eviction 对长上下文有价值，短工作负载不是主瓶颈；
- FlashAttention 提升显著，但属于成熟外部计算优化，不是项目原创核心。

被实验拒绝的优化同样构成贡献边界

静态 RANDOM、盲目 Expert Prefetch、Popular-16、LFRU、Combined 组合、Self-draft Speculative Decoding、NTFS3 mmap 等均出现端到端回退。项目以 Wall Time 和同合同吞吐拒绝局部指标看似更好的方案。

6. 测试指标、意义与比较合同

指标	表示什么	为什么需要
Prefill token/s	Prompt 批量处理速度	反映顺序大范围访存和 Attention
Decode token/s	单 token 持续生成能力	最核心的交互速度与缺页敏感指标
Wall Time	完整任务端到端耗时	防止 Hit Rate 上升但整体反而变慢
cgroup current/peak	系统资源合同	证明 2/3 GiB 硬约束真实生效
RSS/VmHWM	进程常驻工作集	观察进程内存，但不能与 cgroup 直接混算
Major Page Faults	需要存储读取的缺页	反映 mmap I/O 压力
Weight/Expert issued/skipped bytes	实际预取与拒绝的数据量	判断优化是否制造无效 I/O

指标	表示什么	为什么需要
In-flight peak bytes	同时在途的预取峰值	观察突发 I/O 和页缓存挤兑
Expert hit/waste bytes	预测正确/ 错误的实际成功建议字节	比单纯 Expert ID Accuracy 更接近系统成本
Reclaim/Residency counters	机制是否真实触发	避免把随机波动误归因于某个模块
小规模质量检查	量化/Top-k 的输出风险	防止只追求速度而忽略模型质量

所有提升只在 **同设备、同模型、同量化、同缓存状态、同 CPU/内存限制和同 Workload** 内计算。不能把 8 GiB、32 GiB、不同量化和不同 Cache 的 0.08 → 5.16 串成正式 64.5×；HiDevLab 也只是 aarch64 CPU-only 可移植性验证，不是 Ascend NPU 加速。

7. 最有代表性的结果

- **WSL/x86, 8 GiB, 同合同**: Decode 0.08 → 0.43 token/s，验证强内存压力下页管理收益。
- **Mac/Colima, 2 GiB、4 vCPU、no-swap**: 48.41 GB 80B 模型完成推理，验证极限可运行性。
- **RK3588, 3 GiB**: Decode 0.70 → 2.21 token/s；动态页建议把静态 RANDOM 的 0.44/0.26 恢复至 2.84/1.40。
- **Raspberry Pi 5 慢存储**: Shared/Hot 驻留提高 Decode 14.16%、缩短 Wall 6.58%；16 MiB Budget 减少投机 Weight I/O 98.44%。
- **HiDevLab aarch64 CPU-only**: 证明 POSIX 路径可移植，但小模型、内存充足、Warm Cache 下运行时管理成本可能高于收益。
- **32 GiB Warm Cache 的 FlashAttention 消融**: Decode 3.01 → 5.16 token/s，说明权重 I/O 压力下降后瓶颈会迁移到 Attention；该结果属于独立合同和成熟算法协同。

8. 完整贡献表述

一句话完整版

SLIM-ARC 在不改变模型权重和原生 Router 语义的前提下，把 MoE 动态稀疏性转化为阶段、设备和压力感知的操作系统页级闭环：对普通权重与专家执行有界预取，以 Generation Token 将预测和真实 Router 结果精确结算，安全回收错误专家内部页、驻留高复用页，并用统一准入控制有限内存与 I/O；同时通过 Model-owned Runtime、Lease、幂等补丁和结构化证据链，使该机制能在多类端侧设备上安全复现并明确其适用边界。

15 秒答辩版

FlexInfer 主要回答“怎样异步搬后续权重”，SLIM-ARC进一步回答 MoE 和极限内存下“什么时候搬、搬哪些、猜错了怎么办、哪些页值得留下、资源不足时如何收缩，以及不同设备上什么时候反而不应该搬”。

四个不能讲错的边界

- 1. Decode 并非永远适合 RANDOM，最终是阶段与设备可配置策略；
- 2. 2 GiB 是物理工作集上限，不是把 48.41 GB 模型压缩成 2 GiB；
- 3. 当前统一 Budget 对 Weight/Expert 接入最完整，KV 尚未达到同等级逐轮 Hard Admission；
- 4. 跨合同数据不能相乘，HiDevLab 未完成 NPU Kernel 加速。

9. 核心代码地图

文件	作用
slim-arc-runtime.*	Model-owned Runtime、Lease、激活与析构屏障
slim-arc-prefetch.*	普通权重/专家预取、Router 状态、Generation、指标、Hot Cache
slim-arc-unified-scheduler.*	阶段预算、压力采样和 Weight/Expert 额度下发
slim-arc-page-range.*	页范围向内/向外对齐、合并与溢出检查
slim-arc-expert-reclaim.*	错误专家识别和安全 DONTNEED 计划
slim-arc-expert-residency.*	压力滞回、浪费 EWMA 和候选准入
slim-arc-expert-transition.*	在线跨层专家转移统计与 Top-k 预测
slim-arc-kv-eviction.*	KV Hot/Warm/Cold、滑窗与 Offload 原型
scripts/apply-slim-arc.py	幂等注入 llama.cpp、校验最终集成状态
scripts/agent/edge_agent.py	常驻模型端点、短上下文、输出预算和受限 Shell